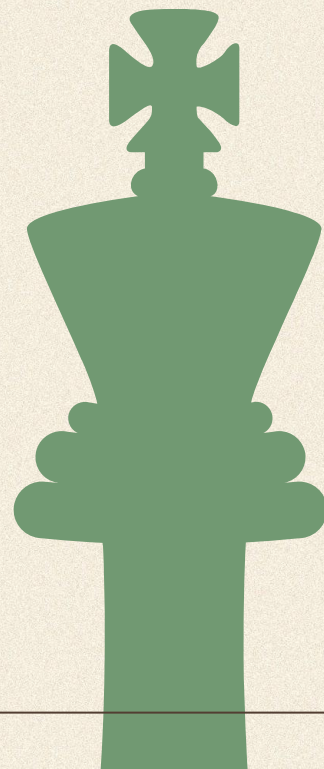
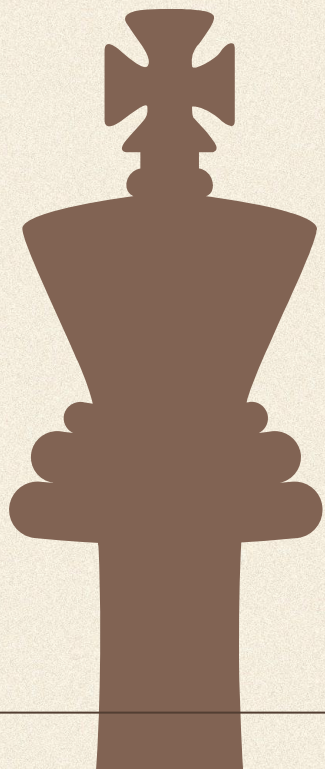


Joseph Hoane Chess Robot

Jordan Rios, Brayden Hayworth,
Connor Tynan, Evan Litzer

IOWA STATE
UNIVERSITY

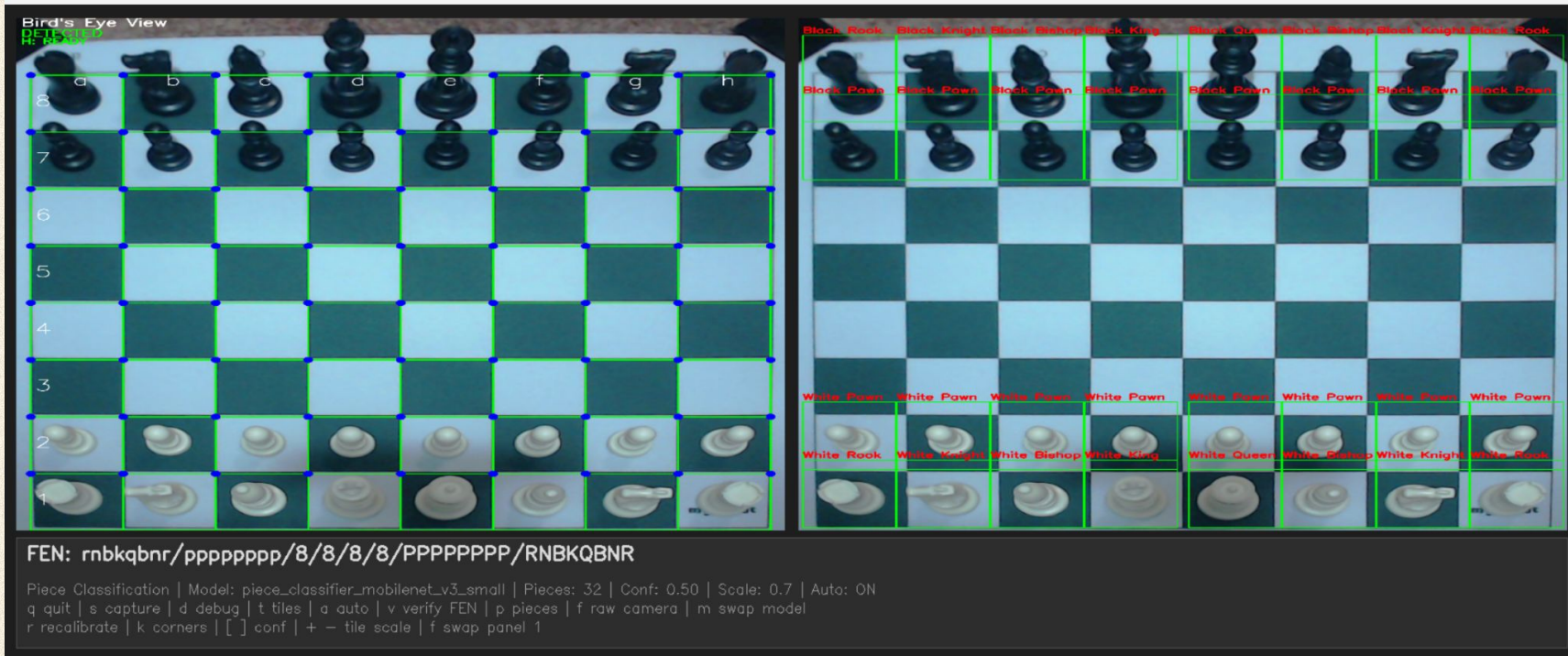


Main Changes to Vision Module

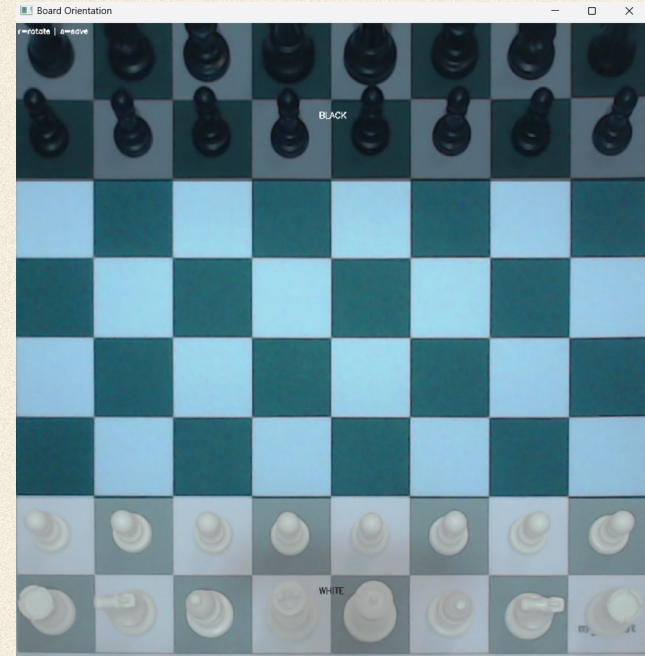
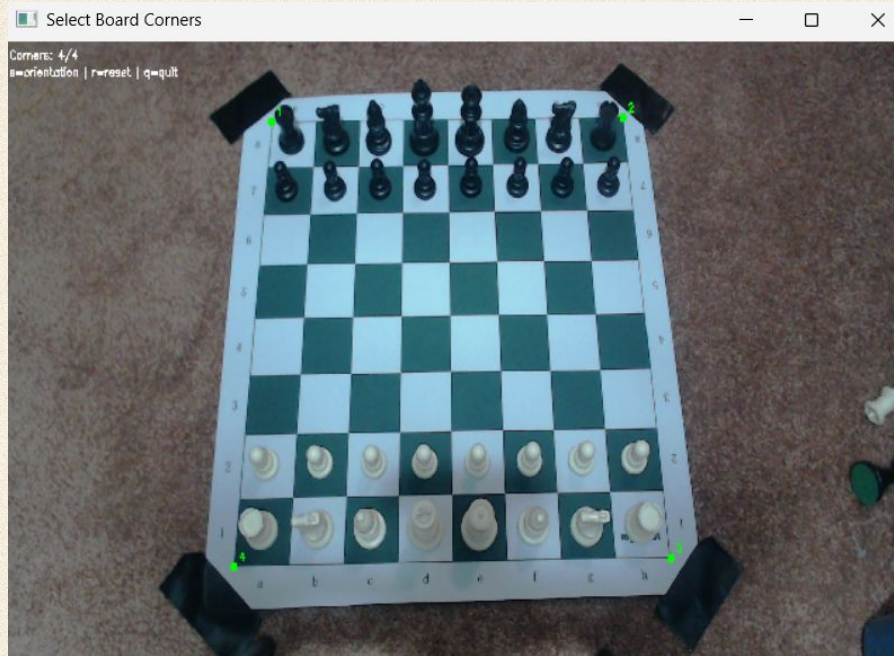
- Trained a piece classification model using a pre trained model (currently mobilenet_v3_small) and are using transfer learning via PyTorch to train it.
- We now pass in 64 images into the pieces classifier and which classifies each square and builds the FEN notation.
- Changed the camera resolution to 1920x1080 (was 1280x720) and tile capture size to 128x128 (was 64x64).
- Created an adjustable dynamic tile cropping to combat the distortion caused by the camera angle.
- Created a training image capture system and auto labeling after providing a FEN notation of the board of the current board state.
- Made changes to the manual board corner calibration to allow for rotating the rank and file of the board.
- Cleaned up GUI and reduced clutter

Current Vision GUI

Live Chess Detection - Multi Panel View

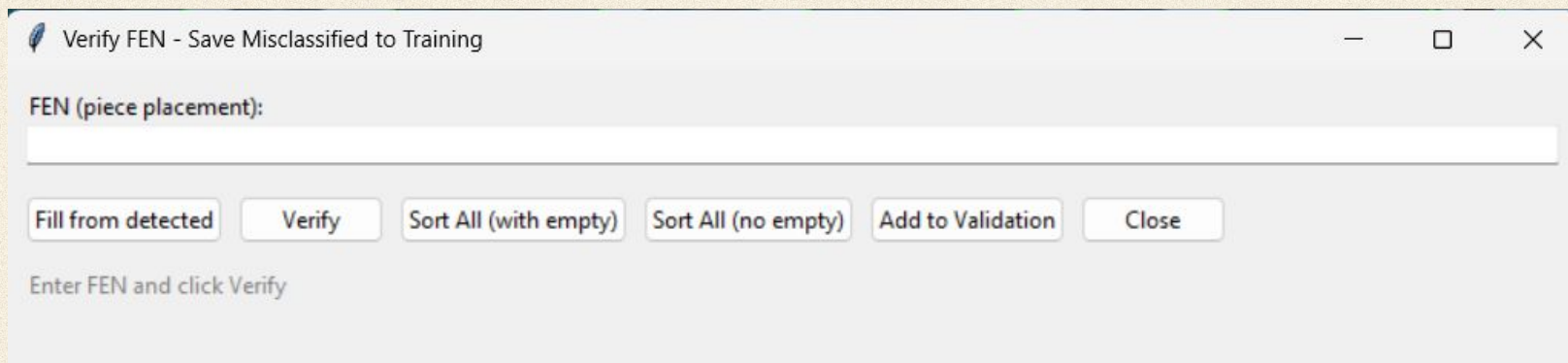


Vision Corner Calibration



Model Training (Transfer Learning)

- We currently have a labeled training data set of 942 tile images, and a validation set of tiles 256.
- We also train on each image multiple times with random variations in lighting and the images being flipped or cropped differently.
- Also added a GUI pop up that helps collect training data which allows you to save the current board position as training data in various ways.



Game Engine Pipeline

Game Engine is the brain of the system

1. Read FEN from vision model, checked/validated for legality and difference from previous FEN
2. Turn change and move detection
3. Move is validated and committed to board – objects are updated
4. GameStateMachine transitions to new state → ROBOT_THINKING
5. Stockfish is called, best move is returned in UCI
6. State machine transitions to ROBOT_MOVING
7. Move sent in json to robot planner





State Machine

Engine is always in one of four states

WAITING_FOR_OPPONENT → Opponent Moved → **ROBOT_THINKING**

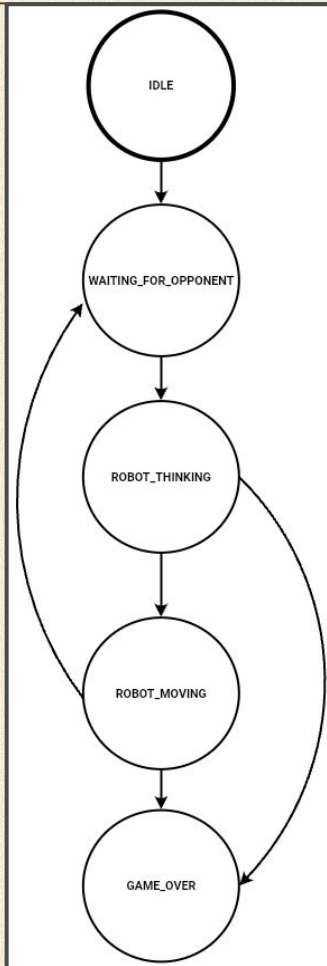
ROBOT_THINKING → move computed → **ROBOT_MOVING**

ROBOT_MOVING → arm confirmed → **WAITING_FOR_OPPONENT**

Game ends → **GAME_OVER**

- State transitions are enforced in GameStateMachine class
- Main loop ticks every 50ms checking for state machine updates
- Nothing happens unless the state machine allows it





FEN Stability + Validation

FENReader reads JSON file outputted by vision model, and **FenStability** filters the noisy readings.

- Game loop calls vision model api each tick for vision updates.
- FenStability maintains buffer of last N amount of FENs using sliding window filtering.
 - Majority: M of last N || Strict: Last N → Stable FEN update
 - Stable FEN is only updated if it was not the same before
- Stable FEN is then stored in the BoardState object
- FEN is then validated for legality, preventing any illegal moves
 - This is an issue that need to be removed.

BoardState & TurnManager

Board State: Stores important board information

- Wraps python-chess library
- Tracks current position, full move and FEN history, current legal moves, and whose turn it is.
- All components read from BoardState object.

Turn Manager: Bridges vision input and Board State

- Decision layer between raw FEN input and game logic
- Compares piece placements before and after a move is made, then determines turn change and state transition.
 - If stable FEN changes, compares new and old FEN and filters through moves
 - If legal move is found, transition to state and change turn.

Stockfish + Robot Handoff

Stockfish runs as child subprocess, communicate with stdin/stdout with UCI

- Skill level and time limit hardcoded for adaptive gameplay.
- If Stockfish faults, then fall back to first legal move available in position
- Move returned in UCI format → 'e2e4' → from_square, to_square

Robot Handoff

- Writes move information to robot_commands.json.
- Includes move location, moved piece, and special move tupe information



```
{  
  "from_sq": "h4",  
  "to_sq": "f6",  
  "moved_piece": "q",  
  "captured_piece": null,  
  "promotion_piece": null,  
  "special": null  
}
```


Robotics: Pipeline

- **Two systems:**
 - Motion Planner
 - Visualization/Controller
- Splitting the logic up allows for easier migration to the real robot after testing in the simulator is complete.
 - Motion planner and visualization are independent, planner sends joint position targets on state change.
 - Visualization simulation and real life controller both take the data in the same way, reducing the refactoring necessary when moving to the real robot.



Robotics – Motion Planner

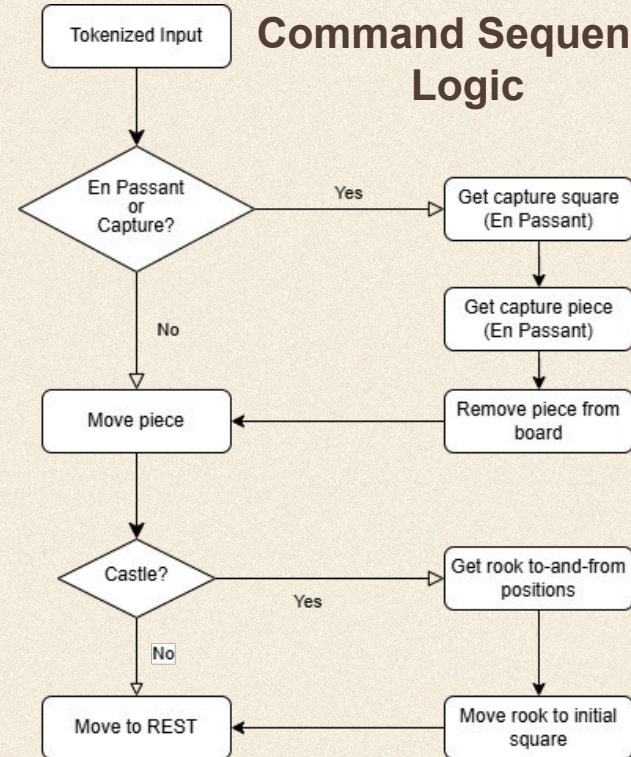
- **Board Configuration:** Configures board distance from arm base and board dimensions to construct rank and file coordinates
- **Coordinate Transformation:** Mapping algebraic notation to Cartesian workspace coordinates (x, y, z).
- **Path Planning:** Collision-free z -axis clearance for piece transit.
- **Input Parsing:** Move string is tokenized into
 - From square
 - To square
 - Captured piece
 - Promoted piece
 - Special action
- Information gets passed to command constructor

Robotics – Motion Planner

Movement Sequence:

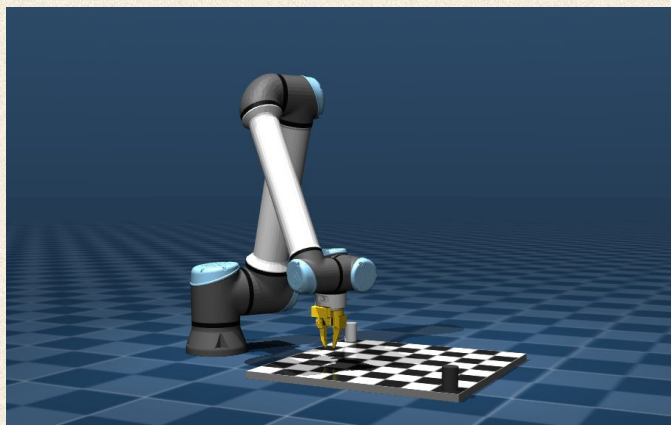
- Starts at rest position →
- Moves to high position over initial square →
- Lowers to piece (depending on piece height) →
- Grab piece →
- Lifts to high position →
- Moves to high position over target square →
- Lowers to place piece →
- Release piece →
- Lifts to high position →
- Back to resting position

Command Sequence Logic



Robotics: Visualization vs Controller

- Mujoco Simulation
 - Fixed time step PID Control
 - GUI with arm model and collidable pieces
 - Plain Python, read/write to target joint position array



- UR10e Controller
 - Continuous control
 - Input target motor positions, robot keeps position itself
 - ROS 1, send motor position topics to the robot (Publisher), read joint positions (Subscriber)

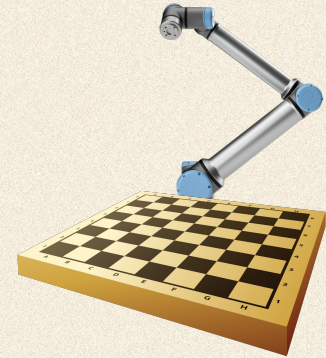
Pipeline Orchestrator

- All subsystems set up their own communication channels, so a overarching program isn't required for the system to function.
- For convenience, a orchestrator script can run all systems at once, ensuring the launch configurations are set up properly to be compatible with each other.
- All systems have a `--auto` flag to signify listening to communication channels, but each system can still be tested independently with manual input by omitting `--auto`



Next Steps

- ROS communication with real life UR10e controller
- Robot arm calibration step, knowing where the board is relative to robot in 3D space
- Make piece classifier more resilient to changes in lighting.
- Relaxed game engine logic, no continuity requirement from vision module.
- Move from simulator to physical robot.



The image features five stylized chess pieces. A large brown King piece is centered in the foreground. Behind it, a green Queen piece is positioned at the top center. To the left and right of the Queen are two brown Rook pieces. In the bottom right corner, a green Pawn piece is visible. The entire scene is set against a light beige background with a thin brown border and black diamond-shaped corner markers.

Questions ?